Universidad CENFOTEC

Escuela de Ingeniería de Software

Código del curso: SOFT-10

Nombre del curso: Estructuras de Datos

Sección: _____ **Periodo:** C1 2026

Docente facilitador: Bryan Hernández Sibaja

CONSIGNA DE PROYECTO

1. Datos generales de la actividad

Fecha de entrega:	29 de Marzo, 2026 11:59 PM	**Valor porcentual:**	10%
Formato de entrega: *(PDF, DOCX, ZIP, video, código, etc.)*	RAR	**Puntaje total:**	100
Individual:	**Grupal:** X		

2. Instrucciones generales *(estas instrucciones pueden ser adaptadas según las especificaciones del curso o de la actividad)*

1. Lea cuidadosamente las instrucciones de la actividad, en caso de tener alguna duda puede consultar con el docente.
2. Esta actividad se desarrolla de manera individual o grupal (de acuerdo a lo especificado en los datos generales), cualquier intento de plagio será sancionado de acuerdo con el reglamento académico vigente.
3. Al completar la actividad, debe subir la solución en la plataforma Moodle en el formato, tiempo y espacio indicado por el docente.

3. Objetivos o competencias del curso que se evaluarán en la actividad de aprendizaje

Objetivo general del curso	Desarrollar la capacidad de diseñar y construir soluciones computacionales integrales mediante el uso combinado de diversas estructuras de datos, seleccionando de forma crítica las más adecuadas según el problema, para modelar información compleja, optimizar procesos y resolver escenarios cercanos al mundo real.
Objetivos específicos que se evalúan	• Comprender los fundamentos teóricos de las principales estructuras de datos (listas, árboles, grafos), identificando

	sus características, operaciones y formas de almacenamiento, para seleccionar la estructura más adecuada según el tipo de problema a resolver. ● Analizar problemas computacionales de mediana complejidad, reconociendo los requerimientos de organización y manipulación de la información, con el fin de modelarlos mediante la combinación de diversas estructuras de datos de forma coherente y eficiente. ● Implementar soluciones de software que integren múltiples estructuras de datos, aplicando algoritmos de búsqueda, ordenamiento y gestión de información, para garantizar un manejo adecuado del rendimiento, la escalabilidad y la mantenibilidad del sistema

4. Descripción de la actividad

Contexto: *El proyecto tiene como propósito que el estudiante aplique de manera integrada diversas estructuras de datos mediante el desarrollo de un Simulador de Teléfono, compuesto por tres módulos funcionales: un reproductor de música, un gestor de archivos y un simulador de red social.*

Cada módulo deberá implementarse utilizando una estructura de datos específica —lista doblemente enlazada circular, árbol y grafo— de manera que el estudiante comprenda sus características, operaciones y aplicaciones prácticas.

El trabajo se desarrollará de forma progresiva en tres avances acumulativos, donde cada entrega deberá integrarse con las anteriores para conformar un único sistema funcional, aplicando principios de programación modular, manejo dinámico de memoria y buenas prácticas de desarrollo.

Instrucciones:

1. El proyecto se desarrollará en tres etapas obligatorias:
    a. **Avance 1 (5%) – Reproductor tipo Spotify**: Implementar un simulador de reproductor de música utilizando una lista doblemente enlazada circular, que permita agregar, eliminar, buscar y navegar canciones hacia adelante y hacia atrás, simulando reproducción continua.
    b. **Avance 2 (10%) – Gestor de Archivos**: Desarrollar un sistema de carpetas y archivos basado en una estructura de árbol, que permita crear directorios (carpetas), insertar documentos (TXT), navegar entre niveles y realizar recorridos jerárquicos. Todo esto, simulando un espacio real en una carpeta en la computadora.
    c. **Avance 3 (15%) – Simulador de Red Social**: Construir un módulo de red social empleando un grafo, donde los nodos representen usuarios y las aristas representen relaciones, permitiendo operaciones de conexión, y recorridos.

2. En esta segunda etapa del proyecto, los estudiantes deberán implementar un Gestor de Archivos, simulando el comportamiento básico de un sistema de carpetas similar al que existe en los sistemas operativos modernos.
   Este módulo permitirá organizar archivos dentro de carpetas jerárquicas, utilizando una estructura de datos tipo árbol, donde cada nodo puede representar una carpeta o un archivo.
   El objetivo es que los estudiantes comprendan cómo los árboles permiten representar estructuras jerárquicas, así como aplicar operaciones de inserción, navegación y recorrido.
   Este módulo deberá integrarse posteriormente con los demás módulos del Simulador de Teléfono.

   Implementación de la estructura de datos

   El sistema de archivos deberá implementarse utilizando una estructura de árbol.

   <u>Nodo sugerido (struct)</u>

   a. Cada nodo del árbol puede representar una **carpeta o un archivo**.
   b. Campos sugeridos:
      - **nombre** (string)
      - **tipo** (string o enum: "carpeta" / "archivo")
      - **contenido** (string) → solo para archivos .txt
      - **hijos** (lista de nodos hijos)
      - **padre** (puntero al nodo padre)

   c. En esa medida, la estructura debe permitir representar jerarquías como:

```
/root
      /Documentos
            tarea1.txt
                  tarea2.txt
      /Fotos
                  imagen1.txt
      /Musica
```

3. Operaciones mínimas requeridas

   Los estudiantes deberán implementar al menos las siguientes funcionalidades.
   a. Gestión de carpetas: El sistema debe permitir:
      - Crear nueva carpeta dentro del directorio actual.
      - Eliminar carpeta (solo si está vacía o con confirmación).
      - Listar las carpetas y archivos dentro del directorio actual.

b. Gestión de archivos: El sistema debe permitir únicamente trabajar con archivos tipo .txt.
   Operaciones mínimas:
   - Crear archivo dentro de la carpeta actual
   - Eliminar archivo
   - Leer contenido de archivo
   - Mostrar lista de archivos dentro de la carpeta

c. Navegación entre carpetas: El sistema debe permitir desplazarse dentro del árbol.
   Operaciones mínimas:
   - Entrar a una carpeta
   - Regresar a la carpeta anterior
   - Mostrar la ruta actual

d. Recorridos del árbol: El programa deberá implementar al menos **un recorrido del árbol** para visualizar la estructura completa del sistema de archivos.

   Ejemplo:

```
root
 ├── Documentos
 │    ├── tarea1.txt
 │    └── tarea2.txt
 ├── Fotos
 │    └── imagen1.txt
 └── Musica
```

4. Interfaz del programa: El sistema debe ejecutarse mediante línea de comandos (consola). Además, debe existir un menú principal que permita ejecutar las operaciones del gestor de archivos.
   El programa debe manejar correctamente situaciones como:
   - Directorio/Carpeta vacía
   - Intentar entrar a una carpeta inexistente
   - Eliminar archivos que no existen

5. Entrega de archivo RAR: En un único archivo **.RAR**, incluya obligatoriamente:
   a. Todos los archivos **.cpp y.h** (si aplica)
   b. Debe de venir incluidas las funcionalidades del primer proyecto.
   c. El ejecutable **.exe** (compilado y funcional). Se realizará un rebajo importante a los grupos que no entreguen su archivo ejecutable.
   d. La carpeta o archivos **.txt** que contienen las letras utilizadas en la demo (para que pueda probarse).

5. Rúbrica

Esta actividad de aprendizaje será evaluada mediante la siguiente rúbrica:

Criterios de desempeño	Deficiente (1 punto)	Regular (2 puntos)	Bueno (3 puntos)	Excelente (4 puntos)
Implementación de la estructura de árbol	No implementa la estructura de árbol solicitada	La estructura del árbol es parcialmente correcta o presenta errores en la relación padre–hijo	El árbol funciona pero presenta detalles menores en la gestión de nodos	Implementa correctamente un árbol dinámico con manejo adecuado de nodos, relaciones jerárquicas y punteros
Gestión de carpetas	No permite crear o eliminar carpetas	Permite crear o eliminar carpetas pero con fallos frecuentes	Permite crear y eliminar carpetas con detalles menores	Permite crear, eliminar y listar carpetas correctamente dentro de la jerarquía
Gestión de archivos (CRUD)	No existe gestión funcional de archivos	Solo implementa 1–2 operaciones básicas	Implementa la mayoría de las operaciones con algunos detalles menores	Permite crear, eliminar, leer y listar archivos correctamente
Navegación entre directorios	No permite navegar entre carpetas	Permite navegar solo en algunos casos	Permite navegar con algunos detalles menores	Permite entrar a carpetas, regresar al directorio anterior y mostrar la ruta actual correctamente
Recorrido del árbol	No implementa recorridos del árbol	Implementa el recorrido pero con errores o salida incompleta	El recorrido funciona pero presenta detalles menores en la visualización	Implementa correctamente recorridos del árbol mostrando la estructura jerárquica completa
Robustez y validaciones	El programa falla ante errores o entradas inválidas	Presenta pocas validaciones	Control parcial de errores en algunas operaciones	Controla correctamente errores como carpetas inexistentes, árbol vacío o archivos inválidos
Interfaz por consola	No existe menú funcional	Interfaz poco clara o difícil de utilizar	Interfaz funcional con detalles menores	Menú claro, navegación intuitiva y mensajes comprensibles para el usuario
Estructura del RAR	No permite probar el programa	Estructura desordenada o incompleta	Falta algún archivo menor	Incluye correctamente .cpp, .h, .exe y archivos necesarios para ejecutar el programa